



Dynamic dispatching rule selection for the job shop scheduling problem[☆]

Nuno Marques^{ID*}, Gonçalo Figueira^{ID¹}, Luis Guimarães^{ID¹}

INESC TEC, Faculty of Engineering, University of Porto, Rua Dr. Roberto Frias, s/n, 4200-465, Porto, Portugal

ARTICLE INFO

Dataset link: <https://github.com/nuno-a-marques/drss>

Keywords:

Dynamic scheduling
Job shop
Dispatching rules
Supervised learning
Reinforcement learning

ABSTRACT

Uncertainty is pervasive in modern manufacturing settings. In order to cope with unexpected events, scheduling decisions are commonly taken resorting to dispatching rules, which are reactive in nature. However, rule performance varies according to shop utilisation and due date allowance, which often change in dynamic real-world job shops. Therefore, this paper explores systems that select dispatching rules as conditions change over time, namely periodic and real-time dispatching rule selection systems, which are based on supervised learning and reinforcement learning algorithms, respectively. These types of systems have been proposed in the past but have been further improved in this work by carefully selecting the most relevant state features and dispatching rules. Moreover, by testing both approaches on the same instances, it was possible to compare them and determine the most advantageous one. After the tests, which included a wide array of job shop instances, both periodic and real-time systems outperformed state-of-the-art dispatching rules by over 10% tardiness-wise. Nonetheless, the periodic rule selection approach was more robust across all tests than the real-time approach. These results demonstrate that there is a real incentive for managers to adopt dispatching rule selection systems.

1. Introduction

Managing operations in modern manufacturing settings has become increasingly complex due to more complicated products, shorter life cycles and frequent uncertain events (Yin et al., 2017). To face these challenges, managers are pushing for Industry 4.0, which is expected to grow at a 15% compound annual growth rate until 2030 (Subramanian, 2024). Under this paradigm, production settings are becoming Cyber-Physical Systems where physical and virtual worlds interact seamlessly with the support of state-of-the-art technologies (Xie et al., 2019). These systems can monitor the facilities in real-time with little human intervention, reacting to uncertain events within milliseconds (Khajeh et al., 2022).

Scheduling is critical across several industries. However, scheduling approaches must be aligned with the new manufacturing reality in which there is little time to react to unexpected events (Parente et al., 2020). Therefore, researchers have been exploring real-time scheduling in various real-world applications, such as in the automotive (e.g. Gros et al., 2020), aeronautic (e.g. Zhou et al., 2019), semiconductor (e.g. Govind et al., 2008), consumer appliances (e.g. Zeng et al., 2024) and pharmaceutical (e.g. Bouazza et al., 2019) industries. The majority of

the mentioned works rely on Dispatching Rules (DRs), which have become popular in real-time scheduling because they are reactive and easy to implement (Nguyen et al., 2019). Due to their benefits, DRs have been preferred to exact and metaheuristic methods, which are computationally demanding. Additionally, it has been shown that exact methods do not bring quality improvements over DRs in highly dynamic and uncertain environments (Rajabinasab & Mansour, 2011).

Over the years, researchers designed several DRs for different conditions and optimisation criteria. However, despite their best efforts, no DR dominates all others (Baker, 1984). Instead, DR Selection Systems (DRSSs) that dynamically select the DR according to conditions emerged in the literature (Priore et al., 2018).

Two main types of DRSS have been proposed. One type, selects the rule in real-time, thus called Real-Time Rule Selection (RTRS). The second type selects the DR periodically, thus can be called Periodic Rule Selection (PRS). Up until this moment, there is scant literature comparing these two approaches. Even within each approach, it is unclear from the literature how to define the state set (i.e., the features that describe the state of the shop floor) and the action set (i.e., the DRs that can be selected by the DRSS).

[☆] Nuno Marques acknowledges the support by national funds through Fundação para a Ciência e a Tecnologia (FCT), Portugal [reference: 2021.07619.BD].

Gonçalo Figueira and Luis Guimarães acknowledge the support from the European Union's Horizon 2020 research and innovation programme [TRUST-AI research project, grant agreement number 952060].

* Correspondence to: INESC TEC, Faculdade de Engenharia, Universidade do Porto, Rua Dr. Roberto Frias, s/n, 4200-465, Porto, Portugal.

E-mail addresses: nuno.a.marques@inesctec.pt (N. Marques), goncalo.figueira@fe.up.pt (G. Figueira), lguimaraes@fe.up.pt (L. Guimarães).

¹ INESC TEC, Faculdade de Engenharia, Universidade do Porto, Rua Dr. Roberto Frias, s/n, 4200-465, Porto, Portugal.

Glossary

D3QN	Duelling double deep Q-network
DDPG	Deep deterministic policy gradient
DDQN	Double deep Q-network
DQL	Deep Q-learning
DQN	Deep Q-network
DR	Dispatching rule
DRSS	Dispatching rule selection system
DT	Decision tree
QL	Q-learning
NN	Neural network
PPO	Proximal policy optimisation
PRS	Periodic rule selection
RF	Random forest
RL	Reinforcement learning
RTRS	Real time rule selection
SL	Supervised learning
SVM	Support vector machine

Motivated by the abovementioned gaps, this work presents the following contributions:

- A comparison of both PRS and RTRS. These models can cope with the dynamism of real-life settings due to their execution speed. The performance of the models is assessed in a wide range of dynamic job shop instances, including stress situations where the arrival pattern is deeply affected. Testing PRS and RTRS in the same instances allows for direct comparison, which has not been addressed in the literature. Furthermore, the models were also compared with state-of-the-art DRs, and not only elementary rules, showing gains of over 10% tardiness-wise.
- PRS models are based in different algorithms, namely Random Forest (RF), Neural Network (NN), and Support Vector Machine (SVM). These algorithms are not only trained for classification (as standard in the literature) but also for regression. Decision Tree (DT) is a prevalent algorithm in previous PRS works, although RF is more powerful than DT. Thus, RF is preferred.
- For RTRS, both Q-Learning (QL) and Deep Q-Learning (DQL) are tested. The former is suitable for discrete state spaces, while the latter is suitable for continuous state spaces. By implementing QL and DQL, the performance loss related to state space discretisation is assessed. Existing works in the literature rely solely on a single algorithm.
- A complementary action set composed of state-of-the-art DRs. Most works rely on a few elementary DRs (e.g., first in first out (FIFO), earliest due date (EDD), shortest processing time (SPT), critical ratio (CR), job slack (SLACK)). In this work, a large set of expert and AI-generated DRs, which is present in Table A.12, is tested to ensure the quality and the complementarity of the action set. Evaluating the level of complementarity between DRs is key in DRSS. If the level of complementarity is high, i.e., different instances benefit different DRs (which could be generated by humans or artificial intelligence), then a DRSS is advisable. Hence, rule selection and rule generation are not necessarily alternative approaches, but could indeed be complementary.
- A compact and effective state set is found via correlation analysis. Most works define the state set without any deep analysis of the features involved, often relying on large, unfiltered state sets that become a burden to the DRSS.
- The ‘flusher’, to prevent operations from staying for a long time in the job shop due to being consistently pushed back to the end of the queue by DRs. When the flusher detects an operation that is staying abnormally long in the shop (i.e., beyond a given time limit), it overrules the active DR and schedules the stuck operation instead.

In the following section, the dynamic job shop scheduling problem is presented in detail. Then, in Section 3, the competing DRSS approaches that emerged in the literature are introduced. Section 4 is dedicated to the methodology, where the main components of a DRSS are presented. In Section 5, the comprehensive set of instances that were used in computational tests are described. In the second part of the section, results are presented, firstly by showing how the action and state sets were defined. Afterwards, the performance of the proposed DRSSs is assessed by comparing them against existing DRs. The paper ends with Section 6, where the main outcomes of the paper are highlighted and key managerial insights are presented.

2. The dynamic job shop scheduling problem

In the dynamic job shop scheduling problem, a set of jobs J arrives randomly at the shop to be processed in a pool of machines M . Job arrival follows a Poisson distribution. Each job $j \in J$ has a set of operations O_j that must be processed in a given sequence. Each operation $o \in O_j$ takes p_{jo} units of time to be completed and must be processed in a particular machine non-preemptively. Machines process one operation at a time. The goal is to minimise a given criterion, such as the time taken to complete all jobs (makespan), the time jobs stay at the job shop (flowtime) or the time jobs take beyond their due date (tardiness). In this work, tardiness is optimised because, in practical applications, it is crucial to consider the ability to deliver jobs on time. Moreover, low (or null) tardiness values imply that the job shop is operating efficiently. Finally, DRs struggle tardiness-wise when conditions vary significantly (Baker, 1984).

The role of DRs in the JS problem is to sort the queue of machines when they finish processing an operation. DRs compute a score for each operation in the machine queue according to the DR expression. The operation with the best score is chosen to be processed next.

Over the years, several DRs have been proposed for various optimisation objectives and job shop conditions. Earlier works derived DRs manually, e.g., Anderson and Nyirenda (1990) or Raghu and Rajendran (1993). Sels et al. (2012) compared 25 manually derived DRs regarding their flowtime and tardiness performance. Afterwards, researchers started resorting to artificial intelligence to generate new rules, commonly relying on evolutionary algorithms such as genetic programming (e.g. Hunt et al., 2016; Tay & Ho, 2008) and gene expression programming (e.g. Planinic et al., 2022; Teymourifar et al., 2020). In Table A.12, there is a comprehensive list of DRs that have been proposed in the literature. However, in spite of researchers’ best efforts, no rule clearly dominates others (Park et al., 1997). Regarding tardiness optimisation, researchers faced a great challenge as DR performance crossovers in two main dimensions: utilisation and due date allowance (Baker, 1984).

Utilisation concerns the amount of work that has to be processed in the shop relative to its capacity. It is shaped by the parameter utilisation level, u , which impacts the λ parameter of the Poisson distribution that manages arrivals, as expressed by Eq. (1). In Eq. (1), M stands for the number of machines, $\bar{n}$ is the average number of operations per job, and $\bar{p}$ is the jobs’ average processing time. A high utilisation level renders shops under heavy load, while a lower value will lead to emptier shops.

$$\lambda = \frac{M \cdot u}{\bar{n} \cdot \bar{p}} \quad (1)$$

Allowance is related to the amount of time available to process jobs until their due date. It is commonly defined according to the Total Work Content rule, as proposed by Baker (1984). Under this approach, the due date is defined as a multiple of the job’s total processing time, as expressed by Eq. (2). The allowance factor parameter, a , is that multiple. A higher allowance factor means that there is more time to process a job, i.e., it is a less urgent job. For lower allowance levels,

there is less time to finish a job. Thus it is more likely for it to be tardy if not tackled soon enough.

$$d_j = t + a \cdot \sum_o^{O_j} p_{jo} \quad (2)$$

It has not been possible to find ‘perfectly’ behaving DRs as due-date-related DRs lead to better performance in low utilisation or loose due-dates, and processing time-related DRs are more suitable for high intensive load (Ferreira et al., 2022). Therefore, some DRs that have been proposed aim to balance the relative importance of these two dimensions. For example, Raghu and Rajendran (1993) propose the RR-rule, described by Eq. (3), which relies on the utilisation level, u , as a weight for the two components. In spite of rendering good results in general, the RR rule is still outperformed by the rule proposed by Holthaus and Rajendran (2000), $2PT + NPT + WINQ$, in tight load conditions. As different DRs are suitable for specific shop operating conditions, researchers have proposed systems that dynamically select the active DR as conditions change, the aforementioned DRSS.

$$PT \cdot e^u + PT \cdot \frac{S}{RPT} \cdot e^{-u} + WINQ \quad (3)$$

3. Previous works

Initial approaches to DRSS, such as Wu and Wysk (1989), selected the DR by simulating the performance of several candidate rules every time a decision had to be made (e.g., an operation finished). The simulation step is time-consuming, which deviates from the reactive behaviour of DRs. More recently, Đurasević et al. (2023) explored DR selection by resorting to simulation. Even with more powerful modern hardware, the simulation-based approach was much slower than a DR (3-35x slower). Researchers turned to machine learning to help with the dynamic selection of DRs. Two main approaches have emerged, Periodic Rule Selection (PRS) and Real-Time Rule Selection (RTRS), the former tackled with Supervised Learning (SL) and the latter with Reinforcement Learning (RL). Several authors have explored PRS and RTRS, their contributions are summarised in Table 1, and are going to be discussed in the following subsections.

3.1. Periodic rule selection

Over the years, several SL algorithms were proposed to tackle the PRS problem. Earlier works relied on a single algorithm. For instance, Decision Tree (DT) based approaches were proposed for the Hybrid Flow Shop (Choi et al., 2011), Flexible Flow Shop (Park et al., 1997), and for the Job Shop (Metan et al., 2010). Alternatively, Min and Yih (2003) proposed a NN-based PRS for a flexible flow shop. Finally, Liu et al. (2005) proposed a PRS based on SVM for the Flexible Job Shop. The action sets from these works are composed of elementary DRs, which are simple rules generally consisting of one variable. Examples of elementary DRs are the FIFO, EDD, SLACK, CR, SPT, remaining processing time (RPT), modified operational due date (MOD), and modified due date (MDD) rules. Elementary DRs contrast with Composite DRs, which are more complex, as they combine multiple variables. Composite DRs can be generated by experts (e.g. $2PT+NPT+WINQ$ Holthaus & Rajendran, 2000) or by AI (e.g. $2PT+PT \cdot S^+/RPT+WINQ$ Ferreira et al., 2022). To finish, the PRS works present in Table 1 are not clear about how the features that compose the state set of their systems were selected.

In a later stage, as the DRSS literature matured, authors started comparing the performance of several existing SL algorithms (e.g. Đurasević & Jakobović, 2022; Priore et al., 2018; Shiue et al., 2012). In some cases, ensembles of SL models were proposed.

Shiue et al. (2012) explore several SL algorithms regarding the task of finding the most suitable rule at each scheduling period in a flexible job shop, namely DTs, NNs, SVMs, and Ensembles. The work considers

30 features to describe the state space, so a GA algorithm is used to find the most desirable ones. The models can select rules from a set of well-known elementary DRs, namely, SPT, EDD, SLACK, FIFO and SIO. Computational tests show that the models were over 10% more effective than individual DRs in the optimisation criteria of mean cycle time and number of tardy parts. However, in the throughput criterion results were not as impressive, below 1%.

In Priore et al. (2018), several SL algorithms are proposed for PRS in a flexible job shop. Apart from the already explored DT, SVM, NN and Model Ensemble, this work also includes the k -Nearest Neighbour algorithm. The models can use eight DRs, four routing rules (SPT, number in queue (NIQ), work in queue (WIQ) and lowest utilised station (LUS)) and four sequencing rules (SPT, RPT, EDD and MDD). All routing-sequencing rules possibilities are simulated in various operating conditions to build the training dataset. The flexible job shop condition at each monitoring period is described by seven features, such as machine flexibility, machine utilisation, bottleneck status, allowance factor and work in progress. SL models perform 11% better than DRs when optimising mean tardiness and 4% better when optimising mean flowtime. The model ensemble is the best-performing algorithm.

Đurasević and Jakobović (2022) explore PRS with parallel machines. The work compares different algorithms: NN, DT, k -means, and Naïve Bayes. The candidate DRs are generated using Genetic Programming. The dataset is built by simulating small scheduling instances. After computational tests, DTs perform the best in optimising total weighted tardiness. These tests included non-stationary instances, i.e., instances whose underlying parameters, such as allowance or utilisation, vary during individual replications.

All presented references regarding PRS train classification models. To be clear, these models are trained to receive the state at t and output the DR that the model considers to be the most suitable at t . An alternative, which was explored in this work, is to train regression models. In this case, for each DR, a model is trained to estimate the increase in tardiness of using that DR for any given state. This approach is presented in more detail in Section 4.1.

3.2. Real-time rule selection

Regarding RTRS, one can find several works relying on standard QL, which is more suitable for discrete state spaces. To overcome this obstacle, the authors of those works discretised continuous features in several bins. However, the number of bins must be kept low to reduce the state space to explore. Still, these works rely only on one to two features. Wang and Usher (2005) propose a QL algorithm for the single machine problem relying on two features: the number of jobs in the buffer and the estimation of total lateness. These features were discretised in ten bins. Wei and Zhao (2004) propose QL for the dynamic job shop. The agent relies on a single feature called ‘pressure’, which is based on a ratio between lateness and remaining processing time. This feature is discretised in five bins. Chen et al. (2010) propose a QL agent for a multi-objective dynamic job shop problem. The system relies on a single feature, related to work in progress, which is discretised in eleven bins.

In order to propose DRSS with a higher number of features, researchers started coupling standard QL with clustering as an alternative to binning. For instance, the QL algorithms proposed by Yang and Yan (2009) and Wang (2020) for the job shop rely on four features.

All the mentioned classic QL works are restricted to small state sets. Furthermore, they also rely on compact action set, based on a few elementary DRs. Regarding this matter, Chen et al. (2010) presents the most sophisticated approach as three DRs are chosen from a set of 12 elementary DRs using Data Envelopment Analysis.

More recently, the field of deep learning evolved rapidly, with a strong impact in the RL realm, as several deep RL methods have been proposed for the dynamic job shop scheduling problem. In deep RL methods such as DQL, DDPG, and PPO, the Q-function (or its

Table 1

Overview of past literature on the DRSS topic. References are separated by PRS and RTRS. Within each group, sorting is chronological.

References	Problem	Optimisation objective	State set size ^{a,c}	Action set size ^{b,c}	Type of DRs	Models	PRS models		RTRS	Non-stationary
							Class.	Regr.		
Park et al. (1997)	Flexible Flow Shop	Tardiness	3 (6)	4	Elementary	DT	✓			✓
Min and Yih (2003)	Flexible Flow Shop	Flowtime, Slack, Remaining Time	5	7	Elementary	NN	✓			
Liu et al. (2005)	Flexible Job Shop	Throughput	26	7	Elementary	SVM	✓			
Metan et al. (2010)	Job Shop	Tardiness	(26)	3, 4	Elementary	DT	✓			✓
Choi et al. (2011)	Hybrid Flow Shop	Throughput, Flowtime, Tardiness, Number of Tardy	4	14	Elementary	DT	✓			✓
Shiue et al. (2012)	Flexible Job Shop	Throughput, Cycle Time, Number of Tardy	(30)	5	Elementary	DT, NN, SVM, Ensemble	✓			✓
Priore et al. (2018)	Flexible Job Shop	Tardiness, Flowtime	7	7	Elementary	DT, k-nn, SVM, NN, Ensemble	✓			✓
Đurasević and Jakobović (2022)	Parallel Machines	Tardiness	2–6	10–50	Composite (AI-generated)	DT, k-nn, Naive Bayes, NN	✓			✓
Wei and Zhao (2004)	Job Shop	Lateness-Remaining Time	1	3	Composite (Custom)	QL			✓	
Wang and Usher (2005)	Single Machine	Lateness, Number of Tardy	2	4	Elementary	QL			✓	✓
Yang and Yan (2009)	Job Shop	Tardiness	4	3	Elementary	QL			✓	✓
Chen et al. (2010)	Job Shop	Tardiness, Flowtime	1	3 (12)	Elementary	QL			✓	
Mouelhi-Chibani and Pierreval (2010)	Flow Shop	Tardiness	22	5	Elementary	NN+SA			✓	✓
Luo (2020)	Flexible Job Shop	Tardiness	7	6	Composite (Custom)	DQL (DDQN)			✓	
Wang (2020)	Job Shop	Earliness–Tardiness	4	4	Elementary	QL			✓	
Liu et al. (2022)	Flexible Job Shop	Tardiness	25	4	Elementary	DQL (DDQN)			✓	
Gui et al. (2023)	Flexible Job Shop	Tardiness	20	7	Elementary	DDPG			✓	
Ren and Liu (2024)	Flexible Job Shop	Makespan, Tardiness	8	7	Composite (Custom)	DQL (D3QN)			✓	✓
Wang and Liao (2024)	Job Shop	Tardiness	9	11	Elementary	PPO			✓	
Our work	Job Shop	Tardiness	2, 3 (13)	3 (78)	Elementary and Composite	RF, NN, SVM QL, DQL (DDQN)	✓	✓	✓	✓

^a Refers to the number of features of the selection model.^b Refers to the number of DRs in the rule set (i.e action set) that the models can choose from.^c Values in parenthesis are given in cases where a larger set of features or DRs was filtered before finding the final sets.

equivalent) is approximated by one or more NNs. Thus, these methods are much more suitable for continuous state spaces.

Luo (2020) propose a DQL agent to select the most appropriate DRs at each decision point for the dynamic flexible job shop. The most simple DQL agents use a single NN to approximate the Q-values, which is called Deep Q-Network (DQN). However, the DQL agent from Luo's (2020) approach relies on Double Deep Q-Network (DDQN), i.e., the agent uses two NNs to approximate the Q-function (a more detailed explanation of DDQN is present in Section 4.2). The condition of the job shop is described by features such as machine utilisation, tardiness, jobs' completion rate, and operations' completion rate. The agent can choose between six custom dispatching rules. Luo et al. (2021) extend the previous work by considering multiple objectives, namely machine utilisation and tardiness. The proposed RL agent performs better than benchmark DRs in over 80% of the test instances. Ren and Liu (2024) explore RL agents do not rely on DQN or DDQN, but are based on the Duelling Double Deep Q-Network architecture (D3QN) instead. The D3QN architecture divides the network into two sub-NNs, one for estimating the value of each state and another for estimating the advantage of each state–action pair. Then, the outputs of these two sub-networks are merged to compute the Q-value. The setup in Ren and Liu (2024) is very similar to Luo (2020) in terms of state, action, and instances. After computational tests, it is concluded that D3QN

architecture is superior to DDQN in terms of tardiness. Gains range from 4% to 35% across various instances.

Liu et al. (2022) also propose a DQL approach based on DDQN for the dynamic flexible job shop. However, in contrast to Luo (2020), two RL agents are working together, one responsible for machine allocation and another for operation sequencing. Each agent has its own action set, state set and reward function. Regarding job sequencing, the agent can choose among four DRs (SPT, WINQ, SLACK, CR), while the agent chooses the machine directly for machine allocation. The state set sizes are six and twenty five for machine allocation and job sequencing, respectively. According to tests, the proposed agents outperform several DRs and previously proposed RL agents. To be more precise, the win rate ranges between 47% and 75% in four different scenarios.

Gui et al. (2023) propose an RL agent based on Deep Deterministic Policy Gradient (DDPG) agent for the Flexible Job Shop. The DDPG algorithm is based on an actor–critic RL framework. It is close to DQL as it approximates a Q-function (the critic) but differs from DQL as it also approximates a policy $a^*(s)$ (the actor). The existence of the actor allows DDPG to work with continuous action sets. This is key to Gui et al.'s (2023) approach, as the RL agent does not choose a single DR in each decision point. Instead it outputs the weights that combine the different rules. There are seven DRs available in total, three for machine selection and four for operation sequencing. The state

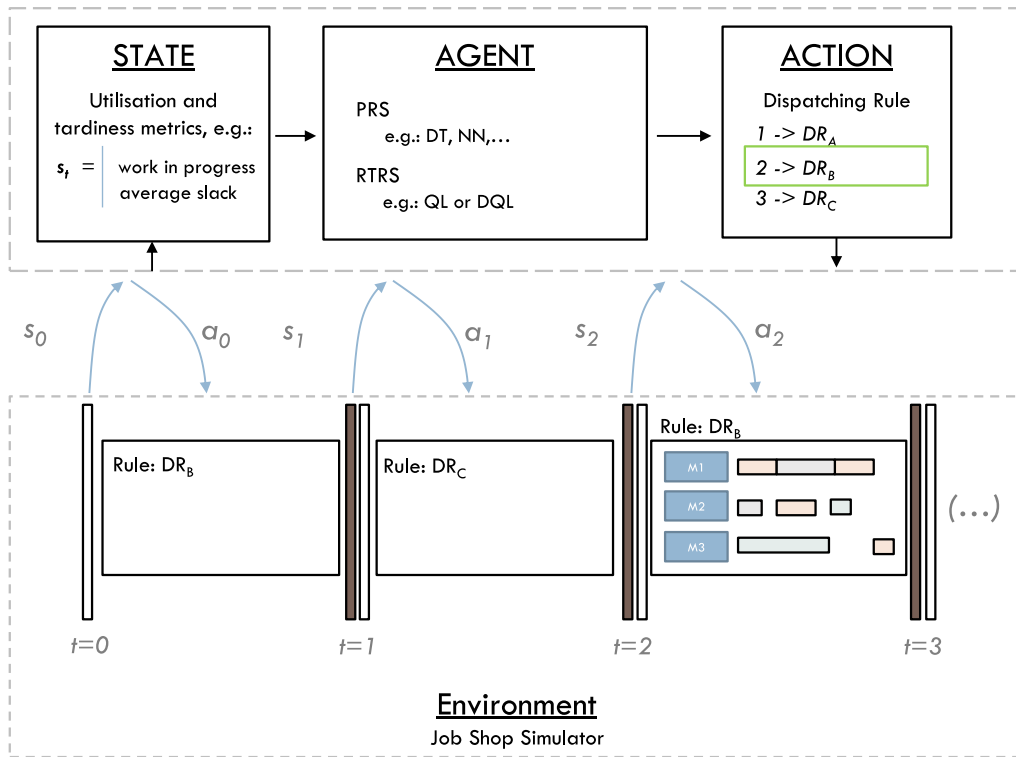


Fig. 1. DRSS main components: agent, state, action (DR) and environment (job shop simulator).

set includes twenty features. DDPG outperformed DQL tardiness-wise in 90% of the instances.

The approach by Wang and Liao (2024) is based on Proximal Policy Optimisation. Compared to DQL, PPO is an RL algorithm that does not approximate a Q-function. It optimises the policy directly instead by resorting to a surrogate objective function. The approach by Wang and Liao (2024) relies on nine state features and eleven dispatching rules. PPO outperformed GP, GA and the best DR in 75% of the instances.

It becomes clear, based on all these references, that deep RL approaches rely on bigger state sets than standard QL.

Finally, opposing all previous authors, Mouelhi-Chibani and Pierreval (2010) state that tackling the DRSS with RL is hard due to the temporal credit assignment problem. Furthermore, SL is also challenging because creating a good training set is not trivial. Thus, he proposes a simulation optimisation technique for the RTRS on a flow shop. An initial NN is established. This NN is not optimised by backpropagation (as in SL), nor by incorporating rewards (as in RL), but through a combination of simulation and Simulated Annealing (SA). The agent relies on 22 hard-coded features and five standard DRs (EDD, SPT, LPT, first in queue (FIQ), and first in system (FIS)). In the end, an improvement of 10% in mean tardiness is observed.

4. Methodology

As depicted in Fig. 1, the DR selection system encompasses four main components, the state and action sets, the agent and the environment.

The state set is the list of features that describes the job shop condition at a given time. Over the years, several state features have been proposed in the DRSS literature, which can be found in Table 2. However, many are interrelated. Thus four distinct groups were defined: machine, pending work, due date and arrival-based related features.

The action set consists of the list of DRs that can be chosen by scheduling agents. Successful action sets are not composed necessarily by the most performant rules overall but by the set of best-performing

rules for different operating conditions. The goal is to find a complementary action set, so that at least a DR is available to the DRSS for each operating condition.

The agent is the model responsible for the mapping between job shop states and suitable DRs. Agent training differs for PRS and RTRS, as explained in detail in the following subsections.

The environment is the job shop simulator. It is responsible for three main tasks. Firstly, generating dynamic job arrivals, according to some predefined parameters. Secondly, allocating jobs to machines when jobs arrive and there are idle machines in the shop, as well as whenever machines become idle. Such is done by computing a score for each operation in the queue of the idle machine, according to the active DR. Subsequently, the operation with the best score is selected to be processed next. Finally, selecting DRs over time as conditions change. In PRS, every time a period ends, the simulator summons the agent, which assesses the state and selects the DR for the next period. In RTRS, the agent is summoned every time a machine becomes idle.

The environment enables us to generate training data for the PRS models. Additionally, it is used to train RTRS models, which learn by interacting directly with the simulator. Lastly, the environment is used to assess the final performance of the models in stationary and non-stationary instances.

A fifth component was added to the DRSS, the ‘flusher’, that prevents orders from staying abnormally long time in the job shop. Such happens because DRs are myopic, particularly the most simple ones. For instance, some rules are processing-time oriented, while others are due-date oriented (Geiger et al., 2006). As a consequence, DRs may leave some operations stuck in the job shop, particularly operations with tail distribution values. The flusher was implemented to fight this phenomenon. It operates as follows: when an operation remains longer than the established time limit, the DR suggested by the agent is overruled, and the stuck operations are scheduled first.

4.1. Agents for periodic rule selection

The agents for PRS were trained via SL, namely RFs, NNs and SVMs. NNs and SVMs have been commonly reported in the literature, as can

Table 2
State set features from the literature, grouped in four groups.

Feature type	Feature	Description	Source
Pending work related	J	number of pending jobs	Shiue et al. (2012)
	PT	processing times of pending operations	Shiue et al. (2012)
	RPT	remaining processing times of pending jobs	Shiue et al. (2012)
	TPT	total processing times of pending jobs	Shiue et al. (2012)
	WINQ	work in the next queue	Shiue et al. (2012)
Due date related	SLK	slack of pending jobs	Shiue et al. (2012)
	TARD	tardiness of pending jobs	Shiue et al. (2012)
	ALLOW	allowance factor	Shiue et al. (2012)
	LAT	lateness of pending jobs	Wei and Zhao (2004)
	EARL	earliness of pending jobs	Wang (2020)
Machine related	U	machine utilisation	Luo (2020)
	A	machine availability	Park et al. (1997)
Arrival rate related	DIST	arrival rate distribution	Đurasević et al. (2023)

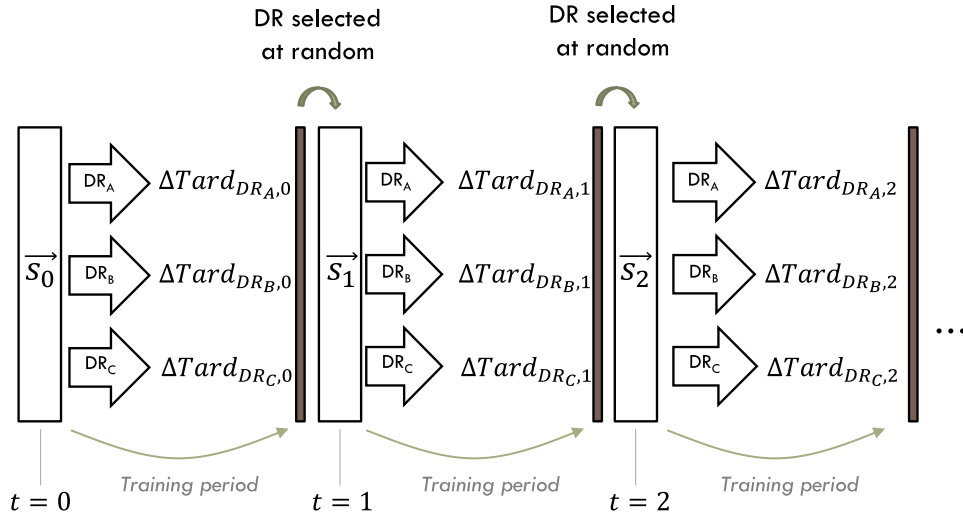


Fig. 2. Overview of the training data generation process for PRS.

be easily observed in Table 1. Previous works frequently implement DTs, which are simple and interpretable. However, RFs were selected instead, as each RF is an ensemble of several individual DTs, which improves accuracy and robustness. In addition to that, each tree of an RF is trained on a sample of the training data, which renders diverse trees, preventing overfitting and promoting generalisation. The RFs were composed of 10 trees, with a maximum depth of 6. The NNs had three hidden layers of 4 neurons each. Sigmoid and Rectified Linear were the activation functions in classification and regression, respectively. Regarding the final layer, the NNs for regression had a single neuron, connected linearly with previous layers. NNs for classification, in turn, had a Softmax layer. In classification, the size of the final layer is equal to the cardinality of the action set. The kernel of the SVM models were radial basis functions.

Regardless of the algorithm, SL requires datasets to train the models. The datasets were built resorting to a technique called multipass simulation (Wu & Wysk, 1989). This technique splits instances into small periods, as Fig. 2 depicts. Then, all rules are tested in a given period from the same starting point (at the beginning of the period), resulting in different levels of tardiness, $\Delta Tard_{DR,t}$. The best-performing rule in that period (i.e., the one that results in the lowest tardiness, as described by Eq. (4)) can be used as a label for a classification algorithm.

$$\text{Selected DR} = \underset{DR}{\operatorname{argmin}} (\Delta Tard_{DR,t}) \quad (4)$$

Alternatively, a regression method can be used, where a label is determined for each DR and corresponds to the tardiness resulting from applying that DR. This tardiness measure can be either the absolute

or relative value. In our case, the label, $\text{Label}_{DR,t}$, is computed as the relative difference between the increment in cumulative tardiness (of all jobs since the beginning of the instance) provided by the DR during this period, $\Delta Tard_{DR,t}$, and the minimum increment in cumulative tardiness recorded by all DRs for that period, $\Delta Tard_{min,t}$, as stated by Eq. (5).

$$\text{Label}_{DR,t} = \frac{\Delta Tard_{DR,t} - \Delta Tard_{min,t}}{\Delta Tard_{min,t}} \quad (5)$$

After evaluating all DRs in a given period, we proceed to the next period. The state at the beginning of this new period can be determined by applying any DRs. Ideally, we should explore the full space of sequences of DRs. However, depending on the number of periods, that can be impractical. Therefore, the multipass approach randomly selects and applies one of the DRs. Note that a random selection, rather than selecting the best DR, promotes a wide range of possible system states.

According to the literature, the length of the training period is a key parameter. It must be long enough to measure the impact of the DR. However, it cannot take too long to capture changes in job shop conditions. So, it is important to try different periods, e.g.: 50, 100, 200, 400, and 800 jobs.

4.2. Agents for real-time rule selection

For RTRS, the agents were trained via RL. In RL, no datasets are necessary as agents improve by interacting with the environment, the job shop simulator in our case. At each decision point t , i.e., every time a machine becomes idle, the agent assesses the state, s_t , and

selects a DR, a_t , according to the agent's policy. The policy is the model responsible for the mapping between states and actions. The impact of an agent decision comes in the subsequent decision point in the form of a reward, r_{t+1} , which is set by the reward function.

QL was the RL method adopted to train the agents. The QL policy is called Q-function. It is responsible for estimating the expected future reward of each state–action pair, $Q(s_t, a_t)$. The Q-function improves by incorporating rewards over time. There is a Q-function update at each decision point, in which state–action pairs are affected by their respective rewards, as shown by Eq. (6). The learning rate, α , defines how much an update modifies the existing $Q(s_t, a_t)$ value. The discount factor, γ , defines the weight that subsequent decision points take in the current update.

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_{t+1} + \gamma \cdot \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right] \quad (6)$$

It is possible to discriminate two types of QL, standard QL and DQL. In standard QL, the Q-function takes the form of a lookup table, thus commonly called Q-table. It is suitable for discrete state spaces. Therefore, if used in problems with continuous state spaces, two solutions exist: state–space discretisation or DQL.

Regarding state–space discretisation, each feature was discretised in five bins. Bins' superior and inferior limits were equal to the maximum and minimum values present in the training dataset, respectively. Note that the logarithmic transformation was applied to the training data prior to limit computation.

In DQL, the Q-function is approximated by an NN, and no discretisation is necessary. Due to DQL's continuous nature, the Q-function update presented in Eq. (6) is not suitable. Instead, the procedure depicted by Algorithm 1 is followed. Briefly, transitions (i.e., $s_t, a_t, r_{t+1}, s_{t+1}$ tuples) are stored in a replay buffer. At each decision point, a mini-batch of transitions is randomly selected from the replay buffer, which is used to train the network using gradient descent.

The DQL algorithm relies on a maximisation over estimated Q-values in the Q-function update step, $\arg\max_{a'} (Q(s_{t+1}, a', \theta))$, which tends to prefer overestimated to underestimated values. As training progresses, instead of accurate estimates, Q-values become overoptimistic due to overestimated values. This phenomenon is particularly pronounced when using the DQN architecture, which relies on a single network to find both the action a' that maximises the Q-value, $\arg\max_{a'} (Q(s_{t+1}, a', \theta))$ and the Q-value itself, $\hat{Q}(s_{t+1}, \arg\max_{a'} (Q(s_{t+1}, a', \theta)), \theta)$. In order to tackle this problem, van Hasselt et al. (2015) proposed the DDQN architecture, which relies on two NNs to approximate the Q-function: the online network, $Q(\theta)$, and the target network, $\hat{Q}(\theta^-)$. The online network is responsible for finding the action a' that maximises the Q-value, $\arg\max_{a'} (Q(s_{t+1}, a', \theta))$, while the target network is responsible for estimating the Q-value, $\hat{Q}(s_{t+1}, \arg\max_{a'} (Q(s_{t+1}, a', \theta)), \theta^-)$, as described by Algorithm 1. The weights of the two networks must be synchronised periodically. In this work, weight synchronisation occurs at the end of an instance.

The input layer of the tested NNs receives the state and the chosen action at time t . Thus, the size of the input changed according to the state set. Note that the action is encoded as a dummy variable, so two extra neurons are necessary besides the neurons for the state set. The NNs had 3 hidden layers with the same size as the input layer. The output of the NN was the estimation of the expected future reward, $Q(s_t, a_t)$.

A central aspect concerning the Q-function updates is the reward function, which in this work is equal for both QL and DQL. According to the literature, the reward function should follow the optimisation criterion (Wei & Zhao, 2004). Therefore, the reward function is tardiness-driven: if tardiness improves, the agent is encouraged;

Algorithm 1 Q-function update, DQL

Randomly sample a set of transitions $(s_t, a_t, r_{t+1}, s_{t+1})$ from Replay Buffer

Set $y_t = \begin{cases} r_{t+1}, & \text{in the last step of the instance} \\ r_{t+1} + \gamma \hat{Q}(s_{t+1}, \arg\max_{a'} (Q(s_{t+1}, a', \theta^-); \theta^-)), & \text{in all previous steps} \end{cases}$

Perform a gradient descent step on $(y_t - Q(s_t, a_t, \theta))^2$ with respect to the network parameters θ

otherwise, it is punished, see Eq. (7).

$$r(t) = \begin{cases} +1, & \text{if TARD at } t+1 \text{ improves from TARD at } t^* \\ -1, & \text{if TARD at } t+1 \text{ worsens from TARD at } t^* \end{cases}$$

*if TARD at $t+1$ is equal to TARD at t ,
use SLK as a secondary criterion, i.e.,
replace TARD by SLK in the conditions above

(7)

RL agents, particularly in the early stages of training, should thoroughly explore the state–action space. As the agent gains experience, it should rely more on its own decisions (exploitation) and less on exploration. The epsilon-greedy exploration strategy became popular to manage the exploration versus exploitation balance and has been used in previous works (e.g. Wang, 2020). Under this strategy, the exploration rate, which controls the percentage of random actions, gradually decreases during training. In this work, the exploration rate started high, at 90%, and linearly decreased to 10%. The rate decreased every time a training cycle finished, i.e., every time the agent went through the training instances. The decrease is 0.004 in DQL and 0.0004 in QL. The rate is lower in QL as it performs a larger number of training cycles due to a faster Q-function update procedure. The agent iterates on the training instances until the time limit of two days is reached.

5. Computational experiments

This section details the instances used for system design and model training, validation, and testing. Furthermore, it shows the tests that support the design decisions for the main components of the DRSS, namely, state set, action set and flusher components. In the final part of the section, the results of the models are presented.

5.1. Instance design

The job shop instances were composed of 20 machines. The rate at which jobs arrive at the shop is defined by the utilisation level, u . Irrespective of the arrival rate, the simulation stops when 2500 jobs are completed. The first 500 jobs are not included in tardiness computation as the job shop takes time to reach a steady state. Jobs' routes vary in size from 2 to 14 operations. Operations' processing times are given according to a uniform distribution that ranges from 1 to $ptMax$. The due date of each job is found based on the job's total processing time and the allowance factor, a , (c.f. Eq. (2)).

Each tuple $u, a, ptMax$ comprises a scenario s from the set of scenarios S . Notice, from Table 3, how the number of scenarios keeps increasing from Training to Validation and from Validation to Test sets to allow assessing model robustness and generalisation capabilities. In PRS, for any given scenario in Training, Validation, and Test sets, we generate 100 instances. As training takes much longer in RL, we generate 25 and 5 instances per scenario for QL and DQL, respectively.

The source code to test PRS and RTRS models in validation and test instances can be downloaded from the following Github repository, <https://github.com/nuno-a-marques/drss>. The Java implementation of the job shop simulator used in this work can be found there. Besides testing the models, the simulator was also responsible for the data

Table 3
Job Shop parameter values considered for Training, Validation and Test datasets.

Parameter	Training	Validation	Test
M Number of machines		20	
J Number of jobs (warm-up)		2500 (500)	
$ O_j $ Number of operations per job		U(2,14)	
p_{jo} Processing time of operations	U(1,ptMax):	ptMax={50,100}	ptMax={50,100,200}
u Utilisation level	0.85, 0.90, 0.95	0.83, 0.87, 0.93, 0.96	0.80, 0.85, 0.90, 0.95, 0.97
a Allowance factor	3, 4, 6	2.5, 3.5, 5, 7	2, 3, 4, 6, 8
S Number of scenarios	18	32	75
$nInst$ Number of instances per scenario	100 for SL 25 for QL 5 for DQL	100	100

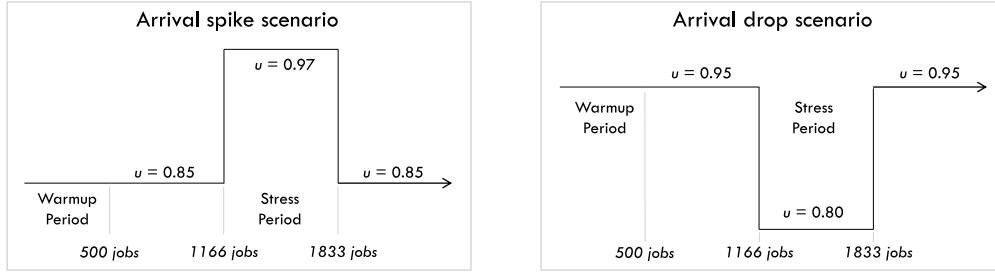


Fig. 3. Non-stationary instance patterns: a sudden arrival of urgent jobs (on the left), and a steep drop in job arrivals (on the right).

necessary to train the PRS models, as well as the training by interaction of the RTRS models. However, the PRS models were not trained in Java but in Python. The Scikit-learn Python library was used to train the RFs and SVMs, while the Keras library was used to train the NNs.

In a deeper effort to test the DRSS against increased uncertainty, the base instances were modified to include stress events. That was achieved by altering the utilisation in the middle of instances. Two different patterns were explored to mimic different stress scenarios that might occur in real life, as depicted in Fig. 3. One pattern aims to simulate a sudden arrival of urgent jobs. The second pattern mirrors a moment in which the job shop has a steep drop in job arrivals. For each type of stress period, a spike or a drop, two versions were tested: one with high urgency ($a=3$) and the other with low urgency ($a=6$). A stress scenario can be described by the tuple $u, a, ptMax$ and u_s , where u_s stands for the utilisation level in the middle part of the instance. There are 100 instances per scenario.

5.2. Model training

Overall, 450 PRS models were trained, namely, RFs, NNs and SVM for both classification and regression. Note that for regression, a model for each DR of the action set must be trained. For the sake of simplicity, the DR-specific models are considered sub-models as they must be used together.

During training, instances were split into periods of different lengths, more precisely, 50, 100, 200, 400, and 800 jobs. Each period renders a different training dataset and, consequently, different models.

Models were generated for each one of the three state sets, which are going to be presented ahead. Finally, for each algorithm (i.e. RF, NN, SVM), learning task (i.e. classification or regression), state set, and training period (from 50 to 800), five alternative models were trained resorting to five different seeds.

Regarding RTRS models, 45 independent runs were conducted overall, as there are 5 initial random policies and 9 combinations of learning rate and discount factor. More precisely, the learning rate in QL took the values 0.2, 0.3, 0.4. For the discount factor, the following levels were tested: 0.7, 0.8, 0.9. In DQL, the discount factor, γ , took the

values: 0.80, 0.85, and 0.90. The learning rate, which in DQL is necessary for the back-propagation process, took the following values: 0.001, 0.005, 0.01. These values are inspired by previous works in the literature, such as Chen et al. (2010) and Luo (2020) for QL and DQL, respectively, as well as preliminary tests. These preliminary tests showed that some learning rate and discount factor values did not render good performance and thus were removed from our analysis. (e.g., in QL, low learning rates such as 0.05 and 0.10, and discount factor of 0.95; in DQL, learning rate levels of 0.005 and 0.0025, and discount factor of 0.95).

Model training was carried out on a computing cluster. Each run was limited to a single core and 1 GB of RAM memory. The nodes are composed of Intel E5-2450 CPUs. The operating system is Linux-based. When training RTRS models, we allowed the search to run up to two days, although most of the best models were found in the first hours. PRS training is substantially faster, as it converges in less than fifteen minutes. The execution time of the policies in each decision point takes, on average, half a millisecond for the slowest algorithm, thus being compatible with real-time applications.

5.3. Evaluation score

Different scenarios render tardiness values of different magnitudes. Therefore, when assessing the performance of the models, one cannot rely on tardiness values directly. For example, a high utilisation level and low allowance factor scenario, e.g.: $u=0.97$ and $a=2$, would lead to higher tardiness values than scenarios with lower utilisation levels and high allowance factors, e.g.: $u=0.80$ and $a=8$, and thus have a greater weight in the final tardiness score. Consequently, models are compared resorting to a normalised tardiness score NT .

The first step to determine NT is to compute the model's performance for each scenario, NT_s . Such is achieved by computing the model's mean tardiness, $\frac{1}{nInst} \sum_{r=1}^{r=nInst} T_{s,r}$, for each scenario divided by the corresponding reference value, RT_s , see Eq. (8). The reference for each scenario is the average performance of the most suitable DR for that specific scenario. The performance score, NT , is the geometric

Table 4

Correlation matrices. Pending work-related features on the left (a), allowance-related features to the right (b).

(a)

	J	PT	RPT
J	-	0.69	0.75
PT		-	0.92
RPT			-

(b)

	SLK	TARD	ALLOW	LAT	EARL
SLK	-	0.91	0.99	1.00	0.56
TARD		-	0.87	0.91	0.17
ALLOW			-	0.99	0.62
LAT				-	0.56
EARL					-

Table 5

QL binning thresholds for each state feature.

Bin	PT: Processing times of pending operations		TARD: Tardiness of pending jobs		EARL: Earliness of pending jobs	
	Inferior limit	Superior limit	Inferior limit	Superior limit	Inferior limit	Superior limit
1	<1096		<12		<6633	
2	1096	1807	12	180	6633	14764
3	1807	2980	180	2440	14764	32859
4	2980	4914	2440	32859	32859	73129
5	>4914		>32859		>73129	

mean of the normalised values across all scenarios, as expressed by Eq. (9).

$$NT_s = \frac{\frac{1}{nInst} \sum_{r=1}^{r=nInst} T_{s,r}}{RT_s} \quad (8)$$

$$NT = \left(\prod_s NT_s \right)^{\frac{1}{|S|}} \quad (9)$$

A score equal to one means that the model is equivalent, on average terms, to use the best rules in each one of the scenarios. Scores below one are possible for those models that outperform the reference values.

5.4. Results

This section shows the results of the various tests performed to define each system component. Afterwards, the performance of the several trained PRS and RTRS models is assessed in a wide array of scenarios, including stress instances.

5.4.1. State set definition

The first step regarding feature selection consisted in removing some features according to the following assumptions:

- Firstly, features should have a global scope. A DR is to be chosen for the entire job shop, so the decision should not rely on information from a single queue (Rajabinasab & Mansour, 2011). Consequently, the DRSS should not depend on a feature such as work in the next queue;
- Information about the future is more important than past information. Therefore, features such as machine utilisation, machine availability, total processing times of pending jobs, and arrival rate distribution are deemed less relevant;
- Two main drivers impact DR performance in job shops, utilisation and due date allowance (Baker, 1984). Thus, at least a feature for each dimension is necessary.

In order to find the most interesting state sets, features were grouped into pending-work and allowance-related features. Subsequently, correlations were computed between features from the same category, as shown in Table 4.

In the pending work-related features group, high correlation values are observed. Therefore a single feature is deemed fit to model the utilisation level of the job shop. The feature ‘PT: processing times of pending operations’ was the selected one as it is the most common in the literature. Two features were chosen to model the allowance

level of the job shop: the ‘TARD: tardiness of pending jobs’ and the ‘EARL: earliness of pending jobs’. These features were chosen due to the low correlation between them, 0.17. No other feature was deemed necessary, as the ‘TARD: tardiness of pending jobs’ feature displays a high correlation with the remaining features. All in all, the proposed DRSS rely on three possible state sets:

- SS_1 : PT & TARD;
- SS_2 : PT & EARL;
- SS_3 : PT & TARD & EARL;

The logarithmic transformation was applied to all features to ease the convergence of PRS and RTRS models during training. Moreover, this transformation was also used to find the limits of the bins for the QL-based systems, please find them in Table 5.

5.4.2. Action set definition

To define the action set several DRs from the literature were tested in the training instances, please confer Table A.12 for the full DR list. It became clear that each DR has preferential operating conditions. In fact, three zones emerged in the utilisation-allowance state space, as depicted by Table 6.

- low allowance, dominated by rules S9, S8 and SHA3;
- high allowance and high utilisation, dominated by the RR rule;
- high allowance and low utilisation, the most scattered zone in space, where rules H1, H2, H4, OZ1, AND2 and PL3 emerged.

The goal is to find an action set with performant rules for different shop conditions. Thus, several candidate action sets were defined by selecting one DR from each zone. There are 18 possible combinations in total. The best performing one was the S9, RR, and H2 set, with a NT score of 1.036. The S9, RR, H4 and S9, RR, AND2 sets were close performers with a score of 1.039. The other DR combinations presented scores above 1.050. All the DRSS proposed in this work relied on the S9, RR and H2 set of rules, as it was the best performing one.

5.4.3. Flusher definition

In order to define the thresholds at which the flusher is activated, the RR, S9, and H2 DRs were simulated in the training instances. After this step, the most suitable DR for each instance was found, as well as the maximum time an operation remained in the queue. Simulation showed that when the most suitable DR was used, no operation remained longer in the queue than 7500 units of time for instances with $ptMax = 50$ or 14000 units of time for instances with $ptMax = 100$. Conversely, when using an under-performing DR, some operations

Table 6

Best performing DR for different utilisation levels u , allowance factors a , and processing times $ptMax$.

(a) $ptMax=50$						(b) $ptMax=100$					
$a \setminus u$	0.8	0.85	0.9	0.95	0.97	$a \setminus u$	0.8	0.85	0.9	0.95	0.97
2	SHA3	SHA3	SHA3	S9	S9	2	SHA3	SHA3	SHA3	S9	S9
3	S9	S8	S9	S9	S9	3	S9	S8	S9	S9	S9
4	AND2	S9	S9	S9	S9	4	H4	S9	S9	S9	S9
6	H2	H4	RR	RR	RR	6	OZ1	H2	RR	RR	RR
8	PL3	H2	H1	RR	RR	8	OZ1	H2	AND2	RR	RR

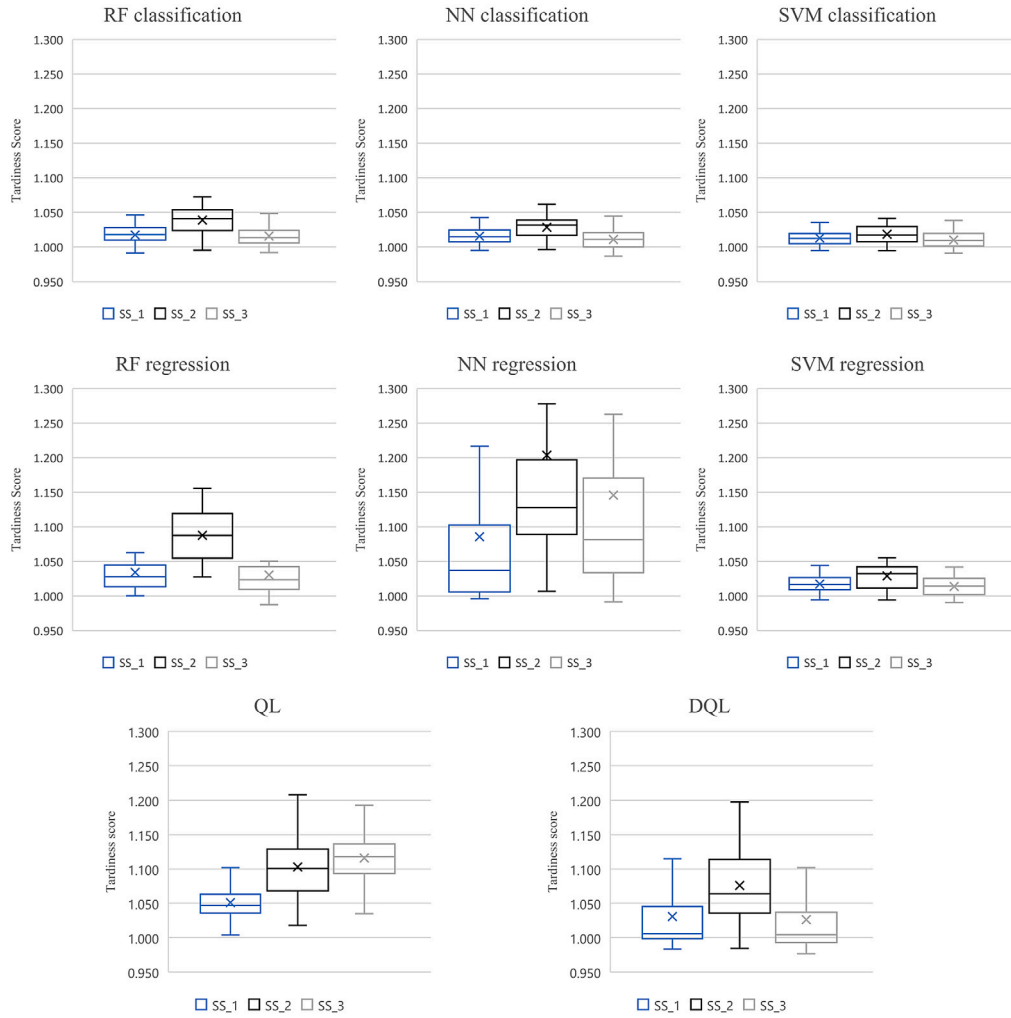


Fig. 4. Performance of the PRS and RTRS models in the validation instances. PRS models are grouped by algorithm, state set, and by learning task. RTRS are grouped by algorithm and by state set.

would be ‘stuck’ and stay much longer in the queue, e.g. 25000 units of time for instances with $ptMax = 50$ or 40000 units of time for instances with $ptMax = 100$.

The activation threshold for $ptMax = 200$ instances was 28000 units of time. This value was found through extrapolation.

5.4.4. Supervised learning models for periodic rule selection

First of all, the impact of the state set on the performance of the PRS models is analysed by resorting to Fig. 4. Data is grouped by algorithm, learning task, and state set. Plots show that models that rely on SS_2 render worse performance than SS_1 and SS_3 -based models, this conclusion is backed by Kruskal–Wallis tests for a significance level α of 5%. Moreover, it is also observable that performance decreases when going from classification to regression, this conclusion is supported by

Mann–Whitney U tests ($\alpha=5\%$) that compare classification models and their regression counterpart. These tests can be consulted in more detail in Appendix B.

As mentioned, PRS models were trained for different training periods ranging from 50 to 800 jobs. However, the frequency at which the DRSS selects DRs does not need to be equal to the length of the training period. For example, a model could be trained in a dataset based on periods of 800 jobs and be deployed in a system that selects DRs at a frequency of 50 jobs. Table 7 shows the best results were found when the training periods were longer (400 or 800 jobs), and the DR selection frequency was shorter (w50 or w100). The larger training periods allow for a better estimation of the impact of DRs, and the shorter selection periods allow for a more reactive behaviour.

Table 7

Tardiness score of the best performing PRS model for each decision frequency and training period combination, according to validation performance. The three best performances are underlined.

		Decision frequency				
		w50	w100	w200	w400	w800
Training period	w50	1.002				
	w100	1.010	1.020			
	w200	1.000	1.010	1.021		
	w400	0.994	1.001	1.012	1.024	
	w800	<u>0.989</u>	<u>0.995</u>	1.003	1.009	1.032

Table 8

NT_r scores of a SVM-Reg model for the several utilisation and allowance combinations. The values in each cell are the average for the different $ptMax$ levels.

$a \setminus u$	0.8	0.85	0.9	0.95	0.97
2	1.00	1.00	1.00	0.97	0.93
3	0.99	1.00	1.00	0.97	0.93
4	0.90	1.00	1.00	0.95	0.93
6	1.15	1.01	1.02	1.05	1.01
8	1.01	0.95	1.02	1.03	1.07

Finally, Table 9 shows the performance of the best training period, DR selection frequency and state set combination for each algorithm-task pair in validation and test instances. According to results, RF-Reg, NN-Class and SVMs perform very well in validation and test instances, showing good generalisation capabilities. The tardiness score is around 1.00, which means the agent can effectively adapt to different load conditions. RF-Class displays a generalisation problem. In spite of a good tardiness score in validation, at par with the best models, in test instances performance deteriorates substantially. Finally, the NN-Reg models' validation performance is the worst registered among DRSSs and they do not generalise well. Irrespective of the algorithm-task combination, the proposed PRS models show significant improvement over using a single DR. Overall, SVM-Reg performed better than other combinations.

Table 9 reinforces the previously mentioned conclusions. To start, the best models prefer longer training periods (400 or 800 jobs) and shorter DR selection frequencies (w50). Furthermore, no algorithm-task combination relies on SS_2 .

The results presented so far are aggregated across the various utilisation and allowance levels. When discriminating by utilisation and allowance, it is observed that models behave differently across the various scenarios. In Table 8, as an illustrative example, the detailed performance of an SVM-Reg model is shown. Taking as a reference the three zones established in Table 6:

- In the low allowance zone, DRSSs can match or outperform the reference DRs for this zone, which are the S9, S8 or SHA3;
- In the high allowance and high utilisation zone, models slightly underperform the reference rule (RR, in this case). This performance reduction occurs because the models, particularly the PRS models, rarely use the RR rule (c.f. Table 10);
- In the low utilisation and high allowance zone, there is no identifiable pattern. One can observe considerable variability because the reference rules in this zone are multiple and change depending on the $ptMax$ level (c.f. Table 6).

5.4.5. Reinforcement learning agents for real-time rule selection

The performance of the RTRS models for the validation instances can be found in Fig. 4. Results are grouped by algorithm and state set. Similar to the results of the SL models, the SS_2 -based models present

worse performance than SS_1 -based models. The performance difference between SS_2 and the other state sets in RTRS is also backed by Kruskal–Wallis tests ($\alpha=5\%$). However, SS_3 -based models behave differently when used by QL or by DQL. SS_3 -based QL models perform worse than SS_1 and SS_2 , while SS_3 -based DQL models clearly outperform SS_2 , and (visually) seem to outperform SS_1 slightly, although the difference is not statistically significant.

From Fig. 4, it is also observable that DQL performs better than QL in validation instances. The performance difference between QL and DQL is corroborated by Mann Whitney U tests ($\alpha=5\%$). Moreover, the best models among RTRS are also based on DQL. Such is expected as the state features are continuous, which QL cannot tackle directly.

Table 9 shows the performance of the five best RTRS models for both QL and DQL. Models were chosen according to their performance in validation instances and were only considered models for the state set that better suited each algorithm. As stated previously, QL performance is inferior to DQL's in both validation and test instances. Furthermore, for both QL and DQL, there is a steep performance decrease of at least 0.05 when going from validation to test. Despite outperforming SVM models in validation instances, DQL performs worse than SVM in test instances (0.03 difference), as it struggles to generalise. According to Table 10, RTRS tend to use the RR rule more often than the S9, which may be beneficial for validation instances, but can be negative for test instances. Nonetheless, RTRS models perform significantly better than using a single DR. Overall, the proposed DRSSs (PRS and RTRS) perform significantly better than a single DR approach.

So far, the presented results for RTRS are aggregated by utilisation and allowance levels. However, when discriminating by utilisation and allowance, RTRS models behave similarly to PRS models. Therefore, the conclusions presented for PRS regarding this matter remain valid for RTRS.

5.4.6. Performance in non-stationary instances

The performance of the best DRSSs (those present in Table 9) in the non-stationary instances is depicted in Table 11. PRS and RTRS systems outperform the DRs, i.e., present tardiness scores lower than 1.022. In fact, some models outperform DRs in all the tested patterns (e.g.: RF-Reg and NN-class). When the urgency is high, the performance of the models is close to 1.000, as the S9 rule is suitable for the vast majority of the time. The DQL model was the best overall ($NT = 0.988$).

Fig. 5 shows how both PRS and RTRS models select rules over time when stress situations occur. For the case of a sudden spike of arrivals, one can see that the S9 rule, which is suitable for high utilisation scenarios, becomes active shortly after the spike, as expected. Conversely, the right side of the figure shows that when there is a steep job arrival drop, the H2 rule comes into effect after some time. Finally, RTRS leads to a much higher number of rule changes, usually in the hundreds, while PRS is more stable.

6. Conclusion

DRs present varying performance according to utilisation and job urgency conditions. Moreover, in real-world settings, conditions change pretty rapidly due to uncertainty. Therefore, DRs quickly become inadequate for the environment they are deployed. All in all, this work mitigates that problem by proposing systems that select the most suitable DR as conditions change in dynamic job shops.

According to the results, it is possible to assess that the proposed selection systems can significantly reduce tardiness when compared to state-of-the-art DRs. These gains are possible due to a complementary action set and a representative state set, enabling the system to pivot swiftly as utilisation and due date allowance conditions change.

In previous works, two types of selection systems were proposed, PRS and RTRS, but it is unclear which is the most advantageous approach. Tests reveal that both types of systems outperform DRs. Additionally, they perform comparatively between each other, showing

Table 9

Average performance and worst-case scenario (WCS) of the DRs, PRSs and RTRSs models for both validation and test instances. In PRS, only the performance of the best combination of parameters (training period, DR selection frequency and state set) for each algorithm-task is shown. In RTRS, only the performance of the best state set is present for each RL algorithm.

Algorithm	Task	Training period	DR selection frequency	State set	Tardiness score			
					Validation Average	WCS	Test Average	WCS
DR	RR	-	-	-	1.146	-	1.219	-
	H2	-	-	-	1.624	-	1.509	-
	S9	-	-	-	1.102	-	1.213	-
RF	Class.	800	50	SS ₁	0.995	1.002	1.060	1.065
	Reg.	800	50	SS ₃	0.998	1.005	1.000	1.005
NN	Class.	800	50	SS ₃	0.989	0.991	0.997	0.998
	Reg.	100	50	SS ₁	1.023	1.054	1.028	1.034
SVM	Class.	400	50	SS ₃	0.994	0.997	0.997	0.998
	Reg.	800	50	SS ₃	0.992	0.993	0.995	0.995
RL	QL	-	-	SS ₁	1.007	1.014	1.060	1.091
	DQL	-	-	SS ₃	0.981	0.984	1.034	1.068

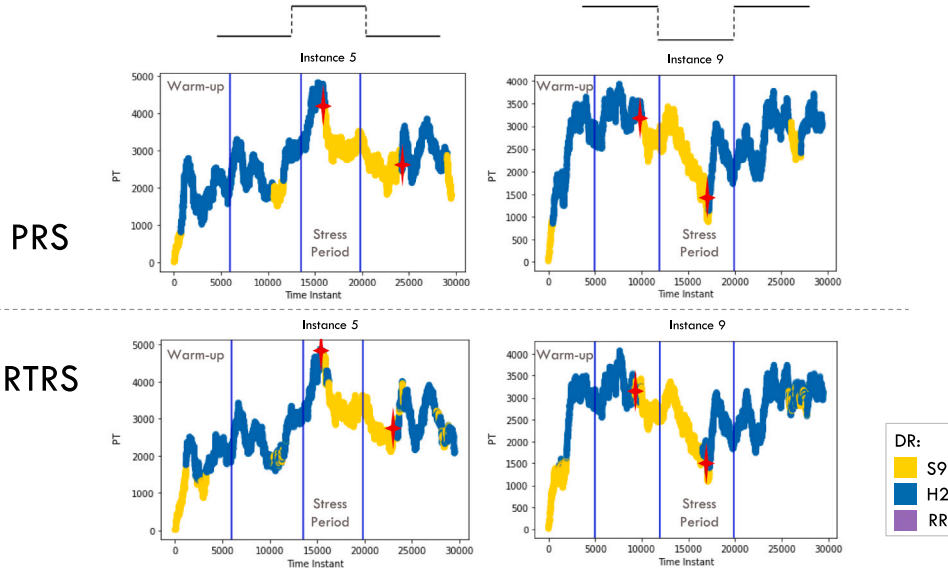


Fig. 5. PRS and RTRS behaviour regarding DR selection over time in non-stationary instances. On the left, an example of a sudden arrival spike. On the right, a steep drop in job arrivals.

Table 10

Number of times each DR was selected by the PRS or RTRS models in the test instances (in percentage). The last column shows the number of times the flusher was activated.

	S9	H2	RR	Flusher
PRS	61.8%	37.1%	0.6%	0.5%
RTRS	47.6%	29.3%	22.7%	0.3%

that both types of systems can effectively learn which DR to use according to the observed conditions. Based on their tardiness scores, which are present in Tables 9 and 11, PRS emerges as the best approach in stationary conditions, as SVM-Reg renders a tardiness score of 0.995

versus a score of 1.034 from DQL. In non-stationary conditions, RTRS slightly outperforms PRS, as DQL scores 0.988 against a score of 0.990 from NN-Class. Compared with PRS, RTRS is quicker to react to changes as it selects the DR every time it schedules an operation, while PRS does it only every 50 to 800 jobs (depending on the period length). Fig. 5 clearly depicts the behavioural difference between PRS and RTRS. RTRS not only leads to a higher number of DR changes but also switches DR sooner than the PRS system. Therefore, these results suggest that non-stationary environments benefit from the reactivity of RTRS, while stationary settings benefit from the consistency of PRS.

From all the conducted experiments, several managerial insights can be drawn. Firstly, managers should analyse if implementing a DRSS is recommendable. PRS and RTRS are suitable for dynamic environments where substantial gains in performance can be made by exploiting complementarity between several rules if used under the right conditions.

Table 11

Performance of the best models in non-stationary instances.

Model	Arrival spike (low urgency)	Arrival spike (urgent jobs)	Arrival drop (low urgency)	Arrival drop (urgent jobs)	Average
RR	1.000	1.117	1.000	1.138	1.062
H2	1.246	1.909	1.215	1.972	1.545
S9	1.041	1.000	1.049	1.000	1.022
RF-class	1.004	1.004	0.999	0.995	1.000
RF-reg	0.999	1.000	0.975	0.995	0.992
NN-class	0.998	0.998	0.973	0.992	0.990
NN-reg	1.020	1.034	0.991	1.012	1.014
SVM-class	1.006	1.000	0.971	0.994	0.993
SVM-reg	1.008	0.998	0.975	0.992	0.993
QL	0.980	1.019	0.961	1.012	0.993
DQL	0.989	1.002	0.967	0.993	0.988

In more stable environments, a single rule may suffice. In stable cases, finding the most suitable DR from the pool of existing DRs or generating a new one (resorting to a hyper-heuristic such as GP) may be more beneficial.

If the implementation of a DRSS is recommendable, then this work provides several hints regarding which type of system to implement: PRS or RTRS. PRS is faster to train as long as a dataset is available. Moreover, it renders a strong and stable performance across stationary and non-stationary instances, even though RF-classification and NN-regression have underperformed. Though RTRS may take longer to converge, it does not require a dataset, it trains by interaction. As a consequence, it can learn continuously over time, something PRS models cannot. However, according to our tests, it does not generalise as well in stationary conditions and leads to a higher number of DR updates, which may be undesirable from a practical standpoint. In addition, RTRS showed a slight edge in cases where spikes or drops occur.

Finally, when the type of DRSS is chosen, this work helps managers to configure the system. For instance, this work shows which features should be monitored and also the most advantageous monitoring period. Moreover, it finds the best-performing DR set and, thus, the one that should be implemented. By implementing a DRSS according to the findings of this paper, managers can rely on a robust and effective system that intelligently selects the most suitable DR over time according to conditions.

After comprehensive computational experiments, PRS and RTRS revealed tremendous potential. Therefore, as future research, it would be interesting to explore these approaches in a real-world application. Real-world problems may require adding constraints to the standard job shop problem, including new features that better fit the real setting and possibly considering multiple and conflicting objectives. Additionally, recent literature, mainly based on RL (e.g. Zhang et al., 2022), aims to dispatch jobs directly and not via DRs. It would be interesting to explore how these emerging approaches compare against the rule selection systems proposed in this work.

CRedit authorship contribution statement

Nuno Marques: Writing – original draft, Validation, Software, Methodology, Formal analysis, Conceptualization. **Gonalo Figueira:** Writing – review & editing, Validation, Supervision, Conceptualization. **Luis Guimares:** Writing – review & editing, Validation, Supervision, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix A. Benchmark DR list

See Table A.12.

Appendix B. Statistical analysis of the results

In this section, a statistical analysis is presented with the goal of validating several of the claims made throughout the manuscript. Non-parametric tests were carried out as key assumptions such as normality were not observed. The significance level α was 5% for all tests. All claims are supported by the tests, except one comparison regarding the performance of different state sets in DQL (c.f. Table B.16).

Claim 1. SS_2 underperforms SS_1 and SS_3 in PRS (c.f. Table B.13).

Claim 2. Classification outperforms regression in PRS (c.f. Table B.14).

Claim 3. DQL outperforms QL (c.f. Table B.15).

Claim 4. SS_2 underperforms SS_1 in RTRS & SS_3 is the worst performing state set for QL, but the best performing state set for DQL (c.f. Table B.16).

Appendix C. RTRS training effectiveness for different parameters

For both QL and DQL, 45 parallel training runs were carried out. These runs differ from each other as they rely on different state sets, learning rates, discount factors, and seeds.

After each training run, the most promising models are selected for further testing. Some runs were more successful than others. Thus, *which parameters affect the training effectiveness the most?*

In QL, according to Fig. C.6, the state set is the dimension that most impacts the effectiveness of training. The learning rate and discount factor do not seem to have an appreciable impact on the number of promising models.

In DQL, as seen in Fig. C.6, the choice of learning rate and discount factor displays a stronger impact on the learning process than in QL. Moreover, the state set impact on DQL training is negligible, but it is the most important dimension in QL. Finally, one of the seeds clearly underperformed compared to the remaining four, showing that DQL training is sensitive to the initial random policy.

Data availability

The instances used in this work can be downloaded from the following Github repository: <https://github.com/nuno-a-marques/drss>.

Table A.12

DR benchmark list. The table shows, with as much detail as possible, where the DRs can be found in the literature. In the rightmost column, the expressions of the rules are present. The ♣ marker identifies the DRs that work by maximising the score.

References	#DRs	Rule source	Rule expression
Anderson and Nyirenda (1990)	3	Equations 3 and 5	AND1: $\max(S/RPT \cdot PT, PT)$; AND2: $\max(CR \cdot PT, PT)$; AND3: CR
Raghu and Rajendran (1993)	1	Section 4	RR: $PT \cdot \exp(u) + PT \cdot S / RPT \cdot \exp(-u) + WINQ$
Holthaus and Rajendran (2000)	1	Equation 2	HR1: $2 \cdot PT + WINQ + NPT$
Tay and Ho (2007)	10	Rule_1 to Rule_10	T1: $TPT \cdot (CT + AT + PT - 3) + (CT \cdot PT + AT \cdot NOP) - (NOP \cdot PT + 2 \cdot PT + CT + 1)$; T2: $(PT + CT + AT + 2) \cdot (RPT + PT + TPT)$; T3: $CT \cdot TPT + 5 \cdot NOP + 3 \cdot AT$; T4: $DD \cdot (AT + TPT + RPT + PT)$; T5: $(TPT + PT) \cdot (CT + AT) + (DD - AT)$; T6: $3 \cdot TPT + (PT / TPT + 1) \cdot (AT + RPT)$; T7: $6 \cdot NOP + PT \cdot CT \cdot (PT + TPT)$; T8: $NOP + 9 \cdot TPT + 4 \cdot PT$; T9: $4 \cdot TPT - 2 \cdot NOP + 3 \cdot DD + 2 \cdot PT$; T10: $DD / TPT + 2 \cdot TPT + PT + DD + AT$
Tay and Ho (2008)	5	Rule_1 to Rule_5	T11: $AT + PT + 2 \cdot TPT$; T12: $7 \cdot TPT + 11 \cdot PT + 12 \cdot (NOP + AT)$; T13: $AT + 2 \cdot PT + 2 \cdot TPT + NOP$; T14: $(AT + DD) + 2 \cdot (AT + TPT) + PT - NOP$; T15: $(AT + DD) + (TPT + PT) - 2 \cdot (AT / NOP)$
Sels et al. (2012)	21	Fig. 1, only DRs for Mean Tardiness and for Large Job Shops	SL1: $2 \cdot PT + RPT + FDD$; SL2: $2 \cdot PT + RPT + S$; SL3: $PT + RPT + S$; SL4: $2 \cdot PT + RPT$; SL5: $2 \cdot PT + RPT + DD$; SL6: $RPT + \max(AT + \alpha \cdot CPT, CT + PT)$; SL7: $RPT + PT$; SL8: $RPT + PT + AVPRO$; SL9: $PT + WINQ + RPT$; SL10: $PT + OPW + RPT$; SL11: $AVPRO$; SL12: RPT ; SL13: DD ; SL14: PT ; SL15: MOD-rule: $\max(AT + \alpha \cdot CPT, CT + PT)$; SL16: $PT + OPW$; SL17: $PT + WINQ$; SL18: $PT + OPW + FDD$; SL19: $PT + WINQ + NPT + WSL$; SL20: $PT + S$; SL21: $PT + WINQ + \min(0, S)$
Hunt et al. (2016)	6	Figures 5, 6, 8, 9, and Expressions from Page 6	H1: $PT / RPT \cdot (CT - DD) \cdot ((NROP + 1) / NROP \cdot AT)^\clubsuit$; H2: $PT / RPT \cdot (CT - DD) / DD^\clubsuit$; H3: if slack > 0 Then $(-PT / (DD + OPW)) \cdot ((CT \cdot (DD - CT + RPT)) / (DD + RPT))$ Else $(-PT / (DD + OPW))^\clubsuit$; H4: $PT \cdot (\min(CT, \max(DD, CT) - RPT) - \max(DD, CT)) / RPT^\clubsuit$; ATC-rule: $(1 / PT) \cdot \exp(-((DD - CT - PT - b \cdot (RPT - PT)) / (k \cdot APT)))^\clubsuit$; CoverT-rule: $(1 / PT) \cdot (1 - S^+ / (k \cdot b \cdot TPT))^\clubsuit$
Ozturk et al. (2019)	4	Rules 8, 10, 11, and SOPT-rule	OZ1: if (Slack > 0) Then Slack/NOP Else Slack * NOP; OZ2: $RPT + ODD$; OZ3: $TPT / (3 + NOP - NROP)$; OZ4: $NROP \cdot (EDD + NROP \cdot ((NROP + TPT) / (RPT - TPT)))$
Teymourifar et al. (2020)	4	Rule 1 to Rule 4	TEY1: PT / TPT ; TEY2: $NROP \cdot (PT + AT) / NOP$; TEY3: $2 \cdot TPT + NROP$; TEY4: $NROP - PT$
Silva (2021)	10	Tables 4.7, 4.8, and 4.9, only the best rules	S1: CR^+ ; S2: $WINQ$; S3: $CR^+ + PT$; S4: $WINQ + PT$; S5: $CR^+ \cdot PT + WINQ$; S6: $CR^+ \cdot PT + WINQ + PT$; S7: $(S^+ + RPT) \cdot (WINQ + PT) / RPT$; S8: $CR^+ \cdot PT + WINQ + 2 \cdot RPT$; S9: $(S^+ + RPT) \cdot (WINQ + PT) / (RPT + WINQ)$; S10: $WINQ + PT + (S^+ \cdot PT) / (2 \cdot PT)$
Zhang et al. (2021)	2	Rules S1 and S2	Z1: $PT + RPT - 1 / PT \cdot (2 \cdot WIQ - 1) - 1 / PT \cdot (WIQ + RPT + (PT + RPT) / (WIQ - 1 + RPT))$; Z2: $NIQ \cdot (PT - 1) \cdot (PT + RPT \cdot \max(PT, WIQ, \max(WIQ, RPT))) + (NIQ + WIQ) \cdot (1 + \max(RPT, NIQ - 1)) \cdot \max(WIQ, RPT)$
Braune et al. (2022)	1	Fig. 5	BR1: $JINQ + PT + 2 \cdot \min(JINQ, PT) + \min(JINQ + PT, APT)$
Ferreira et al. (2022)	2	R1 and R2 rules	FR1: $2PT + PT \cdot S^+ / RPT + WINQ$; FR2: If (Slack > 0) Then $PT + PT \cdot CR^+ + WINQ$ Else $3 \cdot PT + WINQ - (WINQ / PT)$
Planinic et al. (2022)	5	Table 14	PL1: $3 \cdot PT + 2 \cdot S^+ + (S^+)^2$; PL2: $PT \cdot OPW - PT^2 + S^+ \cdot DD + PT \cdot S^+ / (PT - S^+) + PT \cdot (DD / PT + PT + DD) + S^+ \cdot (S^+ + PT) + DD$; PL3: $S^+ \cdot PT + PT$; PL4: $DD - PT + S^+$; PL5: $PT + 2 \cdot DD + S^+$
Shady et al. (2022)	3	TWT, MT and MFT rules	SHA1: $\max(NROP + \max(NROP, PT), PT + WINQ) + \max(\max(S / NROP + PT, PT), NROP - S + WINQ)$; SHA2: $\min((WINQ + S / NROP) / PT, \max(CT, PT \cdot NPT))$; SHA3: $\max(\max(\min(NPT, 2 \cdot WINQ + PT), PT), WINQ - \max(WINQ, PT)) \cdot PT$

The nodes that compose the DRs are the following: AT, arrival time; CT, current time; PT, processing time; NPT, next processing time; APT, average PT of jobs in queue; RPT, remaining processing time; TPT, job total processing time; CPT, amount of work already processed; AVPRO, average PT of job's remaining operations; OPW, operation waiting time in queue; WINQ, work in the next queue; JINQ, jobs in the next queue; NIQ, number of operation in queue; NOP, job total number of operations; NROP, job number of remaining operations; DD, job due date; ODD, operation due date; FDD=AT+CPT, flow due date; S, job slack; S⁺, job slack, if positive; WSL=min(S-WINQ,0), waiting slack; CR=(DD-CT)/RPT, critical ratio; CR⁺, critical ratio, if positive.

Table B.13

Kruskal–Wallis tests for each PRS model and learning task combination, complemented with the respective pairwise comparisons. In the Kruskal–Wallis tests, the null hypothesis is that all medians η are equal, while the alternative hypothesis is that at least one median η is different. All comparisons were deemed statistically significant for a significance level α of 5%.

	Kruskal–Wallis, H_a	χ^2	df	p	DSCF ^a sample 1-sample 2	W	p	Statistical significance
RF classification	$\eta(SS_1) \neq \eta(SS_2)$ or	64.0	2	<.001	SS_1 – SS_2	9.54	<.001	YES
	$\eta(SS_1) \neq \eta(SS_3)$ or $\eta(SS_2) \neq \eta(SS_3)$				SS_2 – SS_3	–9.98	<.001	YES
RF regression	$\eta(SS_1) \neq \eta(SS_2)$ or	103.0	2	<.001	SS_1 – SS_2	12.11	<.001	YES
	$\eta(SS_1) \neq \eta(SS_3)$ or $\eta(SS_2) \neq \eta(SS_3)$				SS_2 – SS_3	–12.64	<.001	YES
NN classification	$\eta(SS_1) \neq \eta(SS_2)$ or	45.8	2	<.001	SS_1 – SS_2	7.19	<.001	YES
	$\eta(SS_1) \neq \eta(SS_3)$ or $\eta(SS_2) \neq \eta(SS_3)$				SS_2 – SS_3	–8.76	<.001	YES
NN regression	$\eta(SS_1) \neq \eta(SS_2)$ or	34.7	2	<.001	SS_1 – SS_2	8.16	<.001	YES
	$\eta(SS_1) \neq \eta(SS_3)$ or $\eta(SS_2) \neq \eta(SS_3)$				SS_2 – SS_3	–4.96	<.001	YES
SVM classification	$\eta(SS_1) \neq \eta(SS_2)$ or	14.2	2	<.001	SS_1 – SS_2	3.68	.025	YES
	$\eta(SS_1) \neq \eta(SS_3)$ or $\eta(SS_2) \neq \eta(SS_3)$				SS_2 – SS_3	–12.64	.001	YES
SVM regression	$\eta(SS_1) \neq \eta(SS_2)$ or	33.2	2	<.001	SS_1 – SS_2	6.07	<.001	YES
	$\eta(SS_1) \neq \eta(SS_3)$ or $\eta(SS_2) \neq \eta(SS_3)$				SS_2 – SS_3	–7.75	<.001	YES

^a Dwass–Steel–Critchlow–Fligner pairwise comparison.

Table B.14

Mann–Whitney U statistical tests to compare models based on classification against their regression counterparts. The goal is to reveal differences between the state sets. In all cases, statistical tests allow one to claim, for a statistical significance of 5%, that classification-based models outperform their regression versions.

	Mann–Whitney U, H_a	Statistic	p	Statistical significance
RF	$\eta(\text{RF_class_}SS_1) < \eta(\text{RF_reg_}SS_1)$	1735	<.001	YES
	$\eta(\text{RF_class_}SS_2) < \eta(\text{RF_reg_}SS_2)$	671	<.001	YES
	$\eta(\text{RF_class_}SS_3) < \eta(\text{RF_reg_}SS_3)$	1889	<.001	YES
NN	$\eta(\text{NN_class_}SS_1) < \eta(\text{NN_reg_}SS_1)$	1764	<.001	YES
	$\eta(\text{NN_class_}SS_2) < \eta(\text{NN_reg_}SS_2)$	483	<.001	YES
	$\eta(\text{NN_class_}SS_3) < \eta(\text{NN_reg_}SS_3)$	808	<.001	YES
SVM	$\eta(\text{SVM_class_}SS_1) < \eta(\text{SVM_reg_}SS_1)$	2173	0.008	YES
	$\eta(\text{SVM_class_}SS_2) < \eta(\text{SVM_reg_}SS_2)$	1694	<.001	YES
	$\eta(\text{SVM_class_}SS_3) < \eta(\text{SVM_reg_}SS_3)$	2371	0.049	YES

Table B.15

For each state set, Mann–Whitney U tests were carried out to compare DQL-based models versus their QL version, and all comparisons were statistically significant for a significance level of 5%.

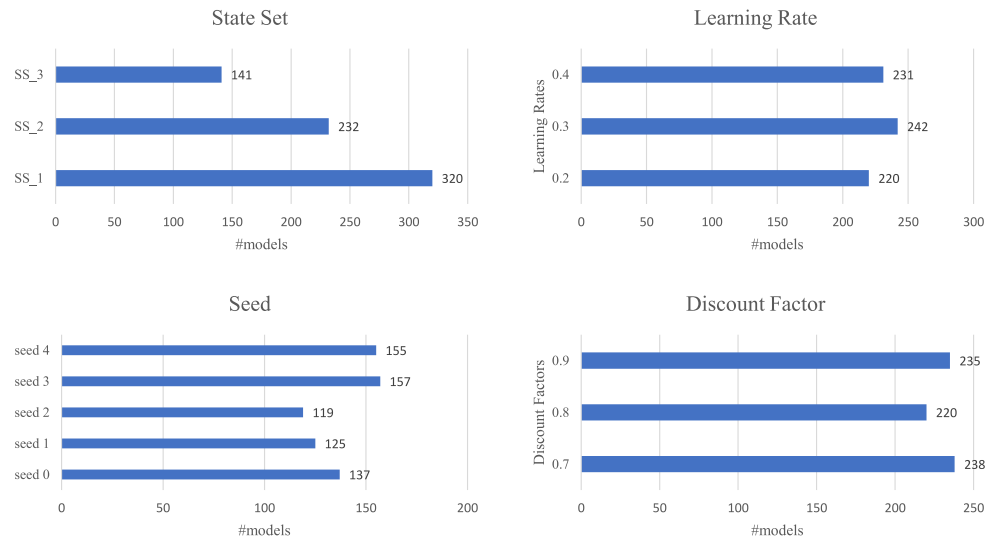
	Mann–Whitney U, H_a	Statistic	p	Statistical significance
	$\eta(\text{DQL_}SS_1) < \eta(\text{QL_}SS_1)$	7689	<.001	YES
	$\eta(\text{DQL_}SS_2) < \eta(\text{QL_}SS_2)$	7887	<.001	YES
	$\eta(\text{DQL_}SS_3) < \eta(\text{QL_}SS_3)$	1157	<.001	YES

Table B.16

Kruskal–Wallis tests, accompanied by Dwass–Steel–Critchlow–Fligner (DSCF), for both DQL and QL. The goal of these tests is to reveal differences between models from different state sets. All tests were statistically significant ($\alpha = 5\%$) except between SS_3 and SS_1 in DQL.

	Kruskal–Wallis, H_a	χ^2	df	p	DSCF Sample 1-Sample 2	W	p	Statistical Significance
QL	$\eta(SS_1) \neq \eta(SS_2)$ or	346.0	2	<.001	SS_1 – SS_2	21.14	<.001	YES
	$\eta(SS_1) \neq \eta(SS_3)$ or				SS_1 – SS_3	21.95	<.001	YES
	$\eta(SS_2) \neq \eta(SS_3)$				SS_2 – SS_3	5.35	<.001	YES
DQL	$\eta(SS_1) \neq \eta(SS_2)$ or	56.7	2	<.001	SS_1 – SS_2	8.31	<.001	YES
	$\eta(SS_1) \neq \eta(SS_3)$ or				SS_1 – SS_3	–2.48	0.186	NO
	$\eta(SS_2) \neq \eta(SS_3)$				SS_2 – SS_3	–9.57	<.001	YES

Q-Learning



Deep Q-Learning

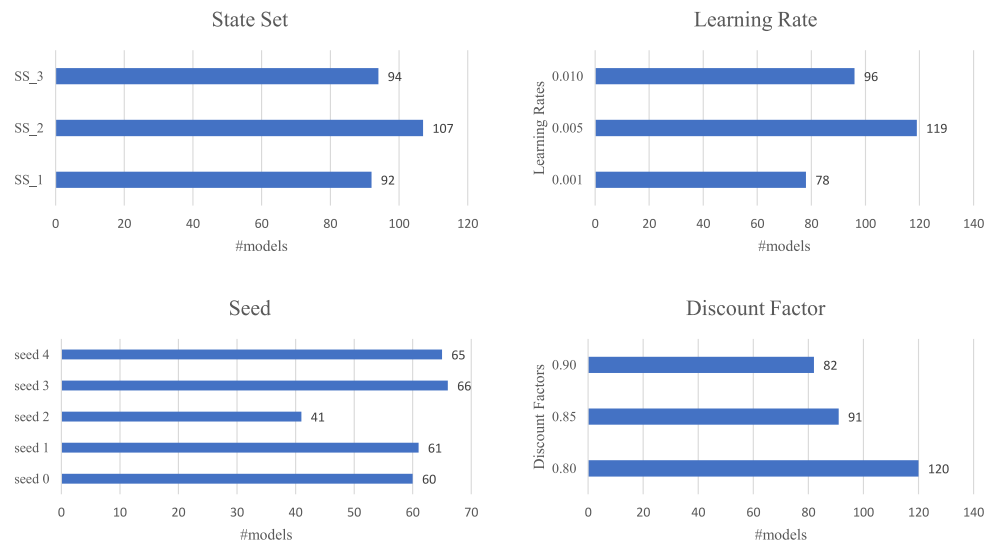


Fig. C.6. Impact of the training parameters (state set, learning rate, seed, discount factor) on the number of promising RTRS models.

References

- Đurasević, M., Gil-Gala, F. J., Planinić, L., & Jakobović, D. (2023). Collaboration methods for ensembles of dispatching rules for the dynamic unrelated machines environment. *Engineering Applications of Artificial Intelligence*, 122, Article 106096.
- Đurasević, M., & Jakobović, D. (2022). Selection of dispatching rules evolved by genetic programming in dynamic unrelated machines scheduling based on problem characteristics. *Journal of Computational Science*, 61.
- Anderson, E. J., & Nyirenda, J. C. (1990). Two new rules to minimize tardiness in a job shop. *International Journal of Production Research*, 28(12), 2277–2292.
- Baker, K. R. (1984). Sequencing rules and due-date assignments in a job shop. *Management Science*, 30(9), 1093–1104.
- Bouazza, W., Hamdadou, D., Sallez, Y., & Trentesaux, D. (2019). Effective dynamic selection of smart products scheduling rules in FMS. *Manufacturing Letters*, 20, 45–48.
- Braune, R., Benda, F., Doerner, K. F., & Hartl, R. F. (2022). A genetic programming learning approach to generate dispatching rules for flexible shop scheduling problems. *International Journal of Production Economics*, 243.
- Chen, X., Hao, X., Lin, H. W., & Murata, T. (2010). Rule driven multi objective dynamic scheduling by data envelopment analysis and reinforcement learning. In *2010 IEEE international conference on automation and logistics* (pp. 396–401).
- Choi, H.-S., Kim, J.-S., & Lee, D.-H. (2011). Real-time scheduling for reentrant hybrid flow shops: A decision tree based mechanism and its application to a TFT-LCD line. *Expert Systems with Applications*, 38(4), 3514–3521.
- Ferreira, C., Figueira, G., & Amorim, P. (2022). Effective and interpretable dispatching rules for dynamic job shops via guided empirical learning. *Omega*, 111, Article 102643.
- Geiger, C. D., Uzsoy, R., & Aytug, H. (2006). Rapid modeling and discovery of priority dispatching rules: An autonomous learning approach. *Journal of Scheduling*, 9(1), 7–34.
- Govind, N., Bullock, E., He, L., Iyer, B., Krishna, M., & Lockwood, C. (2008). Operations management in automated semiconductor manufacturing with integrated targeting, near real-time scheduling, and dispatching. *IEEE Transactions on Semiconductor Manufacturing*, 21(3), 363–370.
- Gros, T. P., Gros, J., & Wolf, V. (2020). Real-time decision making for a car manufacturing process using deep reinforcement learning. In *2020 winter simulation conference* (pp. 3032–3044). IEEE.
- Gui, Y., Tang, D., Zhu, H., Zhang, Y., & Zhang, Z. (2023). Dynamic scheduling for flexible job shop using a deep reinforcement learning approach. *Computers & Industrial Engineering*, 180, Article 109255.
- Holthaus, O., & Rajendran, C. (2000). Efficient jobshop dispatching rules: Further developments. *Production Planning and Control*, 11, 171–178.
- Hunt, R., Johnston, M., & Zhang, M. (2016). *Evolving dispatching rules with greater understandability for dynamic job shop scheduling*. School of Engineering and Computer Science, Victoria University of Wellington.
- Khajeh, S. A., Saberikamarpashti, M., & Rahmani, A. M. (2022). Real-time scheduling in IoT applications: A systematic review. *Sensors*, 23(1), 232.
- Liu, Y.-H., Huang, H.-P., & Lin, Y.-S. (2005). Dynamic scheduling of flexible manufacturing system using support vector machines. In *IEEE international conference on automation science and engineering* (pp. 387–392). IEEE.

- Liu, R., Piplani, R., & Toro, C. (2022). Deep reinforcement learning for dynamic scheduling of a flexible job shop. *International Journal of Production Research*, 60(13), 4049–4069.
- Luo, S. (2020). Dynamic scheduling for flexible job shop with new job insertions by deep reinforcement learning. *Applied Soft Computing*, 91, Article 106208.
- Luo, S., Zhang, L., & Fan, Y. (2021). Dynamic multi-objective scheduling for flexible job shop by deep reinforcement learning. *Computers & Industrial Engineering*, 159, Article 107489.
- Metan, G., Sabuncuoglu, I., & Pierreval, H. (2010). Real time selection of scheduling rules and knowledge extraction via dynamically controlled data mining. *International Journal of Production Research*, 48(23), 6909–6938.
- Min, H.-S., & Yih, Y. (2003). Selection of dispatching rules on multiple dispatching decision points in real-time scheduling of a semiconductor wafer fabrication system. *International Journal of Production Research*, 41(16), 3921–3941.
- Mouelhi-Chibani, W., & Pierreval, H. (2010). Training a neural network to select dispatching rules in real time. *Computers & Industrial Engineering*, 58(2), 249–256.
- Nguyen, S., Mei, Y., Xue, B., & Zhang, M. (2019). A hybrid genetic programming algorithm for automated design of dispatching rules. *Evolutionary Computation*, 27(3), 467–496.
- Ozturk, G., Bahadir, O., & Teymourifar, A. (2019). Extracting priority rules for dynamic multi-objective flexible job shop scheduling problems using gene expression programming. *International Journal of Production Research*, 57, 3121–3137.
- Parente, M., Figueira, G., Amorim, P., & Marques, A. (2020). Production scheduling in the context of Industry 4.0: review and trends. *International Journal of Production Research*, 58, 5401–5431.
- Park, S. C., Raman, N., & Shaw, M. J. (1997). Adaptive scheduling in dynamic flexible manufacturing systems: A dynamic rule selection approach. *IEEE Transactions on Robotics and Automation*, 13(4), 486–502.
- Planinic, L., Backovic, H., Durasevic, M., & Jakobovic, D. (2022). A comparative study of dispatching rule representations in evolutionary algorithms. *IEEE Access*.
- Priore, P., Ponte, B., Puente, J., & Gómez, A. (2018). Learning-based scheduling of flexible manufacturing systems using ensemble methods. *Computers & Industrial Engineering*, 126, 282–291.
- Raghu, T. S., & Rajendran, C. (1993). An efficient dynamic dispatching rule for scheduling in a job shop. *International Journal of Production Economics*, 32(3), 301–313.
- Rajabinasab, A., & Mansour, S. (2011). Dynamic flexible job shop scheduling with alternative process plans: an agent-based approach. *International Journal on Advanced Manufacturing Technology*, 54, 1091–1107.
- Ren, F., & Liu, H. (2024). Dynamic scheduling for flexible job shop based on MachineRank algorithm and reinforcement learning. *Scientific Reports*, 14(1), 29741.
- Sels, V., Gheysen, N., & Vanhoucke, M. (2012). A comparison of priority rules for the job shop scheduling problem under different flow time- and tardiness-related objective functions. *International Journal of Production Research*, 50, 4255–4270.
- Shady, S., Kaihara, T., Fujii, N., & Kokuryo, D. (2022). A novel feature selection for evolving compact dispatching rules using genetic programming for dynamic job shop scheduling. *International Journal of Production Research*, 60, 4025–4048.
- Shiue, Y. R., Guh, R., & Lee, K. (2012). Development of machine learning-based real time scheduling systems: using ensemble based on wrapper feature selection approach. *International Journal of Production Research*, 50(20), 5887–5905.
- Silva, Á. M. F. P. (2021). *Using dimensional aware genetic programming to find interpretable dispatching rules for the job shop scheduling problem* (M.D. thesis), Porto: University of Porto.
- Subramanian, S. (2024). *Themes in machines: How is AI impacting manufacturing?* UBS.
- Tay, J. C., & Ho, N. B. (2007). Designing dispatching rules to minimize total tardiness. *Studies in Computational Intelligence*, 49, 101–124.
- Tay, J. C., & Ho, N. B. (2008). Evolving dispatching rules using genetic programming for solving multi-objective flexible job-shop problems. *Computers & Industrial Engineering*, 54, 453–473.
- Teymourifar, A., Ozturk, G., Ozturk, Z. K., & Bahadir, O. (2020). Extracting new dispatching rules for multi-objective dynamic flexible job shop scheduling with limited buffer spaces. *Cognitive Computation*, 195–205.
- van Hasselt, H., Guez, A., & Silver, D. (2015). Deep reinforcement learning with double Q-learning, december 2015. arXiv preprint arXiv:1509.06461.
- Wang, Y. F. (2020). Adaptive job shop scheduling strategy based on weighted Q-learning algorithm. *Journal of Intelligent Manufacturing*, 31(2), 417–432.
- Wang, Z., & Liao, W. (2024). Smart scheduling of dynamic job shop based on discrete event simulation and deep reinforcement learning. *Journal of Intelligent Manufacturing*, 35(6), 2593–2610.
- Wang, Y.-C., & Usher, J. M. (2005). Application of reinforcement learning for agent-based production scheduling. *Engineering Applications of Artificial Intelligence*, 18, 73–82.
- Wei, Y., & Zhao, M. (2004). Composite rules selection using reinforcement learning for dynamic job-shop scheduling. In *2004 IEEE conference on robotics, automation and mechatronics* (pp. 1083–1088).
- Wu, S. Y. D., & Wysk, R. A. (1989). An application of discrete-event simulation to on-line control and scheduling in flexible manufacturing. *International Journal of Production Research*, 27(9), 1603–1623.
- Xie, J., Gao, L., Peng, K., Li, X., & Li, H. (2019). Review on flexible job shop scheduling. *IET Collaborative Intelligent Manufacturing*, 1(3), 67–77.
- Yang, H.-B., & Yan, H.-S. (2009). An adaptive approach to dynamic scheduling in knowledgeable manufacturing cell. *International Journal of Advanced Manufacturing Technology*, 42(3–4), 312–320.
- Yin, Y., Steckel, K. E., & Li, D. (2017). The evolution of production systems from Industry 2.0 through Industry 4.0. *International Journal of Production Research*, 56(1–2), 848–861.
- Zeng, C., Liu, J., Peng, C., & Chen, Q. (2024). Evolving many-objective dispatching rule pairs for unrelated parallel machine scheduling with sequence-dependent setup times. *Engineering Optimization*, 56(12), 2268–2289.
- Zhang, F., Mei, Y., Nguyen, S., & Zhang, M. (2021). Evolving scheduling heuristics via genetic programming with feature selection in dynamic flexible job-shop scheduling. *IEEE Transactions on Cybernetics*, 51(4), 1797–1811.
- Zhang, Y., Zhu, H., Tang, D., Zhou, T., & Gui, Y. (2022). Dynamic job shop scheduling based on deep reinforcement learning for multi-agent manufacturing systems. *Robotics and Computer-Integrated Manufacturing*, 78, Article 102412.
- Zhou, L., Zhang, L., Ren, L., & Wang, J. (2019). Real-time scheduling of cloud manufacturing services based on dynamic data-driven simulation. *IEEE Transactions on Industrial Informatics*, 15(9), 5042–5051.